Chiron

Documentation Checkpoint and Architecture Consolidation

Progress Report 10 • 1 Feb 2026 — 14 Feb 2026

Master’s Research Progress Update

Fahad Alothman

Progress Report 10

Title: Documentation Checkpoint and Architecture Consolidation
Reporting Window: 1 Feb 2026 — 14 Feb 2026
Submission Context: Master’s Research Progress Update

Abstract

This reporting window was a checkpoint in the strict sense: it did not aim to maximize visible product breadth, but to make the architecture coherent enough for later hardening to proceed without drift. The main achievement was not a new surface feature. It was a clearer definition of what Chiron is becoming: a methodology-driven system in which authored contracts and runtime execution are distinct but tightly mirrored.

This period clarified the three-layer structure that later work depended on. L1 became the layer for methodology-wide topology and registries, L2 for work-unit contract policy, and L3 for workflow grammar and execution structure. In parallel, the distinction between Methodology Design Time and Project Runtime became more explicit. By the end of the window, the project had a stronger way to talk about facts, states, transitions, workflows, artifact slots, and runtime evidence without collapsing them into one generic workflow-builder mental model.

From Stabilization to Architectural Control

The immediate problem entering this window was not lack of ideas. It was excess ambiguity. Earlier work had already accumulated migration pressure, runtime experiments, methodology concepts, and growing design surface complexity. What was missing was a stable narrative that could hold those parts together and prevent later implementation from hardening the wrong abstractions.

That is why this period was documentation-forward without being documentation-only. The project deliberately slowed broad feature growth in order to reduce architectural drift. Repository history in this window shows selective technical landing rather than aggressive expansion, while session evidence shows intense design work around boundaries, workflow contracts, methodology structure, and observability. Taken together, that evidence supports the same conclusion: the project was using this window to codify the model before locking more behavior into code.

The Core Decision: Chiron Is Not Just a Workflow Builder

The most important design decision clarified in this period was that Chiron should not be understood as a generic workflow editor with extra metadata attached. That direction would have made workflows the dominant abstraction and pushed facts, state policy, work-unit identity, and artifact evidence into secondary or ad hoc roles.

Instead, this window clarified a different model. Chiron would be methodology-first. Methodology definitions would specify the structure and policy of work. Work units would carry lifecycle semantics, owned facts, workflows, and durable output contracts. Runtime would then materialize those authored decisions as executions, state changes, fact instances, and artifact evidence. This was not just a naming cleanup. It was a control decision meant to determine what later implementation should preserve and what it should reject.

The Layered Model Clarified Here

This window made the L1/L2/L3 distinction usable.

At L1, Chiron defines methodology topology: work-unit type landscape, methodology-wide facts, agent and dependency registries, and the relationship primitives that shape the method map. At L2, the system defines the contract of a work unit: facts, lifecycle states, transitions, gate condition sets, workflow bindings, and artifact-slot policy. At L3, workflows are described as executable grammar: typed steps, edges, context facts, and step-level behavior.

The significance of this separation is that it prevents unrelated concerns from collapsing into one overloaded editor surface. It also explains why some entities belong to methodology-wide authoring, some to work-unit policy, and some to workflow internals. That distinction later became essential for service boundaries, persistence structure, and UI ownership.

Mirrored Modes: Design Time and Runtime

The same period also clarified the dual-mode structure of Chiron.

Methodology Design Time is where reusable contracts are defined and published. Project Runtime is where a project executes a pinned version of those contracts and records what actually happened. The two modes are mirrored rather than duplicated. Design-time defines

intent, rules, and expected outputs. Runtime records transitions, workflow executions, fact instances, and artifact evidence under those rules.

This distinction made it easier to separate author-facing behavior from operator-facing behavior. Methodology authors define policy, while project operators and collaborators execute and validate under that policy. That separation later informed both product language and runtime architecture.

Entity Framing That Became Foundational

This checkpoint also clarified how the main entities should be understood across the two modes. Facts were no longer treated as one undifferentiated pool. Methodology facts, work-unit facts, and runtime fact instances began to occupy clearer roles. States and transitions were treated as lifecycle policy rather than merely UI status. Gates became declarative readiness and completion checks rather than informal conventions. Workflows and steps became execution grammar rather than the sole authority over meaning.

Artifact handling also became sharper. Design-time artifact slots and templates were treated as contract-owned definitions, while runtime artifact behavior was understood as evidence. In practical terms, the slot model pointed toward a simple but strong distinction between `single` and `fileset`, and the runtime side pointed toward immutable snapshot rows and file-member rows instead of a mutable “latest artifact” record. That framing would later mature into the git-aware evidence model now present in the project, where file members can carry commit/blob references, snapshots can supersede earlier snapshots, and current-state checks can distinguish unchanged state from changed or unavailable state.

Why This Window Matters

Progress Report 10 matters because it reduced the chance of hardening the wrong system. It forced the project to answer a more important question than “what should we build next?” The real question was “what kind of system is Chiron becoming, and which abstractions are allowed to govern it?”

By the end of the window, the answer was significantly clearer. Chiron was becoming a methodology-authoring and methodology-execution system, not a workflow toy, not an agent shell with optional methodology notes, and not a loose collection of orchestration experiments. That clarity created the baseline required for the next window, where architecture would begin to harden into enforceable contract boundaries rather than remain a descriptive model.

Stack Direction at the End of the Window

The technical stack direction remained stable through this period even as the architecture became more explicit:

- **Desktop host:** Tauri [3]
- **Frontend:** React + TanStack Router + shadcn/ui
- **Backend/API:** TypeScript services with Hono + oRPC-aligned surfaces, with the MCP transport direction later converging on Hono-based MCP middleware, Hono itself, and the Model Context Protocol SDK [4]–[6]
- **Execution/control plane:** Effect-first methodology and workflow engine direction, with early agent-runtime direction toward an OpenCode SDK-backed harness boundary [1], [2]
- **Data layer:** Drizzle ORM with a SQLite-first, libSQL/Turso-compatible persistence structure
- **Git/workspace seam:** repository-truth direction that later stabilizes around sandboxed git queries and simple-git-oriented workspace operations [8]
- **Authentication:** Better Auth
- **Runtime/tooling:** Bun + Turborepo [7]

The important signal here is not stack replacement. It is architectural control inside the existing direction. That also applies to persistence: the project increasingly converged on a SQLite-first current horizon because local and desktop execution needed lower operational friction, simpler setup, and faster iteration while methodology correctness and runtime determinism were still the primary goals. The persistence shape was kept compatible with libSQL/Turso-style deployment paths so that later expansion would not require redesigning the model.

Conclusion

Progress Report 10 is best understood as the architectural checkpoint that made later hardening possible. Its main contribution was conceptual precision: L1, L2, and L3 became clearer; design-time and runtime became more explicitly mirrored; and the project gained a stronger vocabulary for facts, transitions, workflows, and artifact evidence.

That may appear less dramatic than a feature-heavy delivery window, but it was strategically decisive. It prevented Chiron from sliding into a workflow-first product shape and established the methodology-first direction that the next two windows would harden and formalize.

References

[1] Effect Authors, “Effect Documentation,” 2026. [Online]. Available: https://effect.website/

[2] OpenCode, “OpenCode Documentation,” 2026. [Online]. Available: https://opencode.ai/

[3] Tauri Programme, “Tauri,” Documentation, 2026. [Online]. Available: https://tauri.app/

[4] Hono Authors, “Hono,” Documentation, 2026. [Online]. Available: https://hono.dev/

[5] Hono, “@hono/mcp,” npm, 2026. [Online]. Available: https://www.npmjs.com/package/@hono/mcp

[6] Model Context Protocol, “Introduction,” 2026. [Online]. Available: https://modelcontextprotocol.io/docs/getting-started/intro

[7] Turborepo Authors, “Turborepo Documentation,” 2026. [Online]. Available: https://turbo.build/repo/docs

[8] simple-git, “simple-git,” GitHub repository, 2026. [Online]. Available: https://github.com/simple-git-js/simple-git/